

Curso de Jogos Digitais

Disciplina de Computação Gráfica OpenGL Moderno (Programmable Pipeline)

Aula 14

Professor: André Flores dos Santos



OpenGL

- O OpenGL que utilizamos até o momento é chamado de **‘OpenGL Immediate Mode (Modo Imediato)’**.
- Vocês devem ter notado no exercício de importação de objetos que quando o número de objetos importado cresceu, começou a acontecer um certo travamento da tela. Isso se dá pela necessidade de haver um armazenamento e tratamento dos dados direto na GPU, por enquanto estamos usando apenas a CPU para fazer todo esse processo.

OpenGL

- O que utilizamos até agora
- OpenGL Immediate Mode (Modo Imediato)
- Uso de glBegin(), glVertex3f(), glEnd()
- Cada vértice enviado para a GPU a cada quadro
- Processamento feito pela CPU

OpenGL

Limitações do modo imediato

- Tela começou a travar com muitos objetos
- CPU sobrecarregada enviando dados a cada frame
- Sem armazenamento de vértices na GPU
- Baixo desempenho em cenas complexas

OpenGL

Usaremos a biblioteca GLFW ao invés de Pygame.

Embora o Pygame seja excelente para aplicações simples e introduções ao OpenGL, ele foi projetado para jogos 2D e não oferece suporte robusto às funcionalidades mais modernas do OpenGL.

O GLFW, por outro lado, é uma biblioteca leve e especializada no gerenciamento de janelas, contexto OpenGL e entrada de teclado/mouse, sendo compatível com as versões mais recentes do OpenGL. Ele permite acessar diretamente o "Programmable Pipeline", essencial para criar shaders personalizados, usar VAOs/VBOs e controlar o pipeline gráfico completo.

Em resumo, **GLFW é mais adequado para projetos com OpenGL moderno porque permite maior controle, melhor performance e compatibilidade com recursos gráficos avançados.**

OpenGL

- **Limitações do Pygame com OpenGL Moderno:**
- Suporte limitado a versões modernas do OpenGL
 - O Pygame não garante compatibilidade com contextos OpenGL acima da versão 3.0, o que dificulta ou impede o uso pleno de shaders, VAOs e outros recursos modernos.
- Gerenciamento de contexto defasado
 - Ele não permite configurar atributos avançados de contexto, como perfil de core (core profile) ou versionamento explícito do OpenGL (por exemplo: OpenGL 4.5).
- Foco em 2D e OpenGL legado
 - Foi projetado principalmente para jogos 2D e OpenGL imediato (glBegin/glEnd), com foco educacional ou prototipação.
- Menor controle sobre a janela e entrada
 - Comparado ao GLFW, o Pygame oferece menos controle direto sobre teclado, mouse, e múltiplos monitores, o que limita o uso em aplicações interativas 3D avançadas.

OpenGL

Por que preferimos GLFW:

- Permite criar contextos OpenGL modernos com precisão.
- Mais usado na indústria e em tutoriais profissionais.
- Foco total em 3D, shaders e pipeline moderno.
- Integra-se melhor com bibliotecas como PyOpenGL e numpy.

OpenGL

Mudança de paradigma

- Uso de **VBOs (Vertex Buffer Objects)**
- Dados enviados **uma única vez** para a GPU
- Uso de **Shaders Programáveis**
- Alta performance e controle completo do pipeline

OpenGL- Comparativo

OpenGL Imediato vs OpenGL Moderno

Característica	Imediato	Moderno
Envio de dados	A cada frame (glBegin/glEnd)	Uma vez (VBOs)
Processamento	CPU	GPU
Desempenho	Baixo com muitos objetos	Alto
Controle	Limitado	Total via Shaders
Uso Atual	Estudo/prototipação	Padrão da indústria

OpenGL- Comparativo

O que aprenderemos a seguir

- Criar **VBOs** (Vertex Buffer Objects)
- Criar e utilizar **Shaders**
- Entender o **Pipeline Gráfico Moderno**
- Renderizar cenas com alta performance!

OpenGL

- OpenGL é uma **API gráfica** que possibilita o **acesso aos recursos gráficos de hardware**.
- A versão mais atual possui mais de **500 funções** que são utilizadas para a especificação de **objetos, imagens, e operações** necessárias para a produção de **aplicações gráficas tridimensionais interativas**.

OpenGL

- OpenGL é concebido como uma **interface independente de hardware**, ou seja, pode ser implementado sobre **diferentes tipos de hardwares e softwares**, independente do **sistema operacional** ou do sistema de gerenciamento de janelas.
 - Nas nossas aulas, a implementação que iremos utilizar é a da biblioteca **PyOpenGL**.
- Por este motivo, o OpenGL **não inclui funções para o gerenciamento de janelas e entrada de dados**. Desta forma, precisamos utilizar recursos a parte para este tipo de gerenciamento.
 - Para estas finalidades, a biblioteca **PyOpenGL** utiliza uma outra biblioteca denominada **GLFW**.

OpenGL

- De maneira similar, o OpenGL não fornece funções para a **descrição de modelos tridimensionais** de objetos ou operações de **leitura** de arquivos de **imagens** (JPG, PNG, etc.).
- Para isso, nós mesmos devemos construir nossos modelos tridimensionais a partir de um pequeno conjunto de primitivas:
 - Pontos, linhas e, principalmente, **triângulos**.

OpenGL

- A **primeira versão** do OpenGL (**OpenGL 1.0**) foi desenvolvida em julho de 1994, e desde então sofreu grandes modificações.
- Além disso, também surgiram **diversas implementações** da mesma para diferentes versões do OpenGL, com o objetivo de **simplificar o desenvolvimento de aplicações**.
- A versão mais atual do OpenGL (chamada de **OpenGL moderno**) difere-se da versão original de maneira bastante significativa, e é com esta versão que iremos trabalhar.

Funções

- De maneira resumida, as principais operações que o OpenGL fornece para a criação de aplicações gráficas são:
 - Operações para a **especificação dos dados** para a construção de formas a partir de **primitivas geométricas** do OpenGL;
 - Execução de vários **shaders** para a realização de cálculos sobre as **primitivas de entrada**, de modo a determinar suas **posições**, **cores**, entre outros atributos de **renderização**;
 - **Converter a descrição matemática** de primitivas geométricas em **fragmentos** associados com sua localização na tela para **rasterização**.
 - Execução do **fragment shader** para cada um dos fragmentos gerados na **rasterização**, o qual determinará a **posição final** e a **cor** de cada fragmento na tela;
 - Executar **operações adicionais** por fragmento, tais como **determinar se um objeto cujo fragmento foi gerado é visível ou não**, ou a **combinação das cores** de um fragmento com a cor atual naquela posição da tela, etc.

Funcionamento

- OpenGL é implementado como um sistema **cliente-servidor**, em que a **aplicação** que desenvolvemos é considerada o **cliente**, enquanto que a **implementação** do OpenGL fornecida pelo **fabricante do hardware gráfico** é considerada o **servidor**.
 - Tanto é que, em algumas implementações, o **cliente** e o **servidor** executam em **diferentes máquinas** conectadas por uma **rede**;
 - Nestes casos, o **cliente** irá **emitir os comandos OpenGL**, os quais serão **transmitidos pela rede** até o **servidor**, o qual irá executar tais comandos de modo a **produzir a imagem final**.

Estrutura

- OpenGL pode se tornar facilmente um programa extenso e complexo. Apesar disso, quase toda aplicação OpenGL geralmente segue a seguinte estrutura:
 - Primeiramente são inicializados os estados associados sobre **como** os objetos devem ser **renderizados**;
 - Em seguida, especificamos **quais** objetos devem ser **renderizados**.

Conceitos

- Alguns conceitos importantes, antes de começarmos a trabalhar na prática:
 - **Renderização** é o processo pelo qual um computador cria uma **imagem** a partir de **modelos**, geralmente especificados **matematicamente** ou através de **estruturas de dados**.
 - OpenGL é um Sistema de **renderização**.
 - Nossos **modelos/objetos** serão construídos a partir de **primitivas geométricas – pontos, linhas e triângulos** – estes **especificados** através de **vértices**.
 - **Shaders** são **funções especiais** executadas pelo hardware gráfico. Podemos pensar em **shaders** como **pequenos programas** que são especificamente **compilados** pela GPU.
 - OpenGL inclui todas as ferramentas de compilação necessárias para traduzir os códigos-fonte dos nossos **shaders para o código que a GPU precisa para executá-los**.
 - Os dois tipos de **shaders** mais comuns são o **vertex shader**, que realiza o processamento por vértice, o **fragment shader**, que realiza o processamento por fragmento gerado pelo rasterizador. **Ambos são obrigatórios em qualquer aplicação OpenGL.**

Conceitos

- A imagem final gerada consiste de **pixels desenhados na tela**.
- Um pixel é o **menor elemento visível** em sua tela.
- Os **píxeis** do seu sistema são armazenados em um **framebuffer**, que é uma **porção de memória** que o **hardware gráfico** gerencia utiliza para “alimentar” o display gráfico.

Instalação e configuração

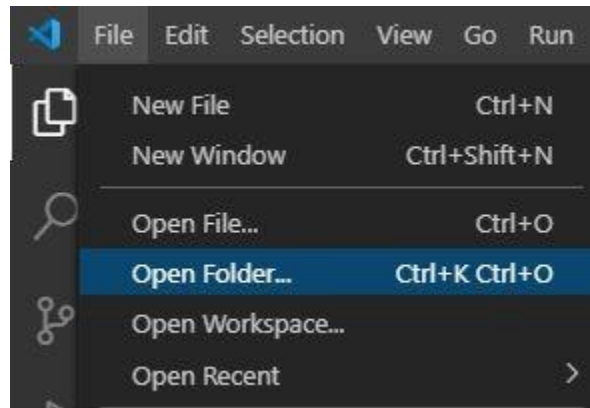
- Nas nossas aulas, para trabalharmos com OpenGL, utilizaremos a IDE Visual Studio Code, na linguagem Python, com a biblioteca PyOpenGL.
- O primeiro passo é realizar o download do Visual Studio Code no site oficial se não tiver instalado:
 - <https://code.visualstudio.com/>

Configuração do ambiente

- Via terminal do Visual Studio Code (pressione ctrl + ' para abrir), instalar os pacotes **glfw**, **pyopengl** e **numpy** do python com o comando:
 - `pip install glfw pyopengl numpy --user`

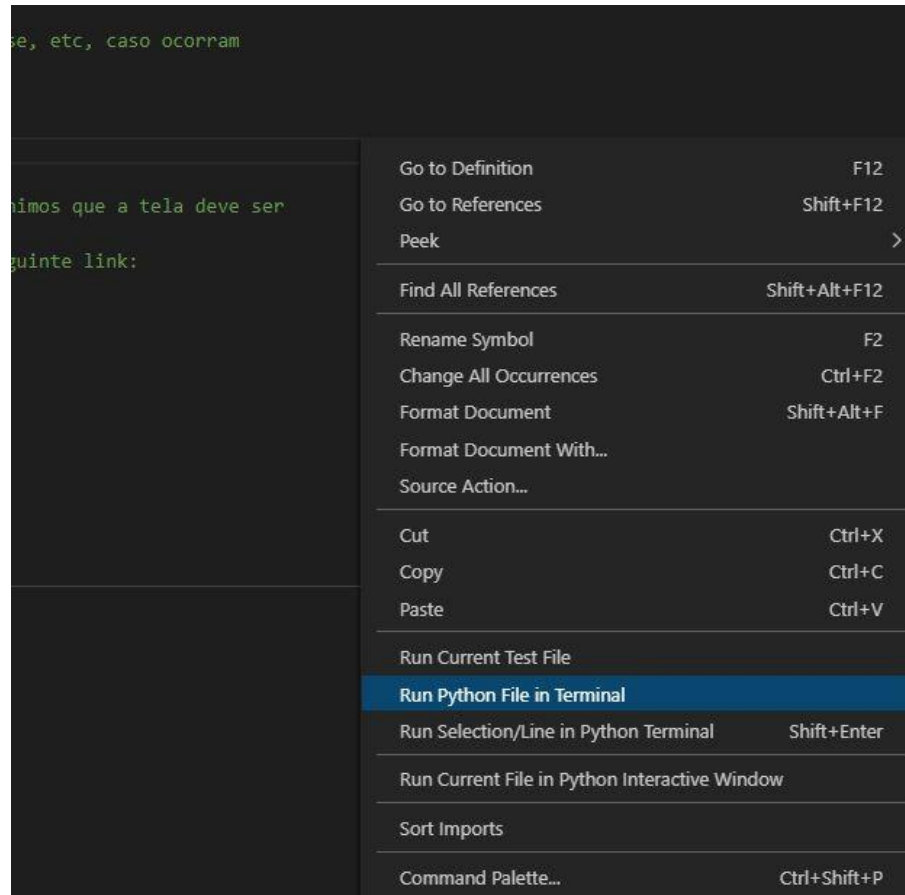
Testando

- Para testar se tudo deu certo, execute o código concedido pelo professor: **Ex1 - Renderização de um triângulo.**
-
- Para executá-lo, coloque-o em uma pasta, e abra a mesma no Visual Studio Code:

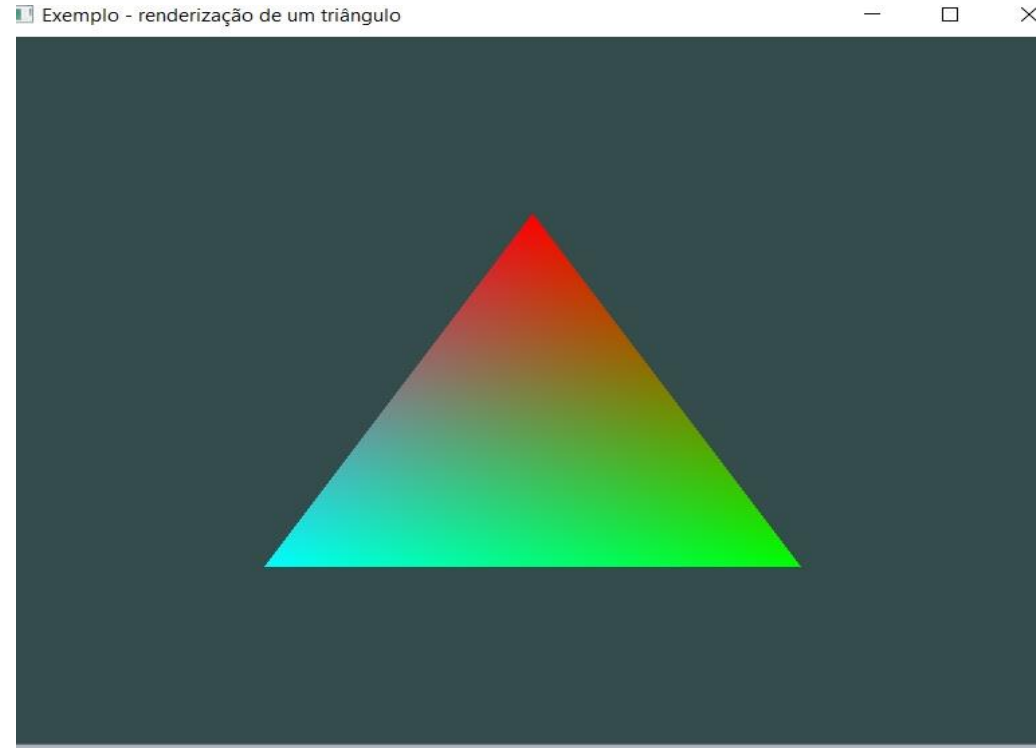


Testando

- Então, clique com o botão direito em cima do código-fonte e clique em **"Run Python File in Terminal"**:



Resultado



Comentários no código

```
Ex1 - Renderização de um triângulo.py > inicializaObjetos
1  # Este exemplo apresenta uma aplicação completa de OpenGL, que renderiza um triângulo na tela.
2  import glfw
3  from OpenGL.GL import *
4  import OpenGL.GL.shaders
5  import numpy as np
6
7  Window = None
8  Shader_programm = None
9  Vao = None
10 WIDTH = 800
11 HEIGHT = 600
12
13 #Função callback que é executada sempre que a janela for redimensionada
14 #Sempre que a tela for redimensionada, salvamos sua nova largura e altura nas variáveis globais acima
15 def redimensionaCallback(window, w, h):
16     global WIDTH, HEIGHT
17     WIDTH = w
18     HEIGHT = h
19
20 def inicializaOpenGL():
21     global Window, WIDTH, HEIGHT
22
23     #Inicializa GLFW
24     glfw.init()
25
26     #Criação de uma janela
27     Window = glfw.create_window(WIDTH, HEIGHT, "Exemplo - renderização de um triângulo", None, None)
28     #Caso não seja possível criar a janela, a GLFW e a aplicação são terminadas
29     if not Window:
30         glfw.terminate()
31         exit()
32
33     #Registramos a função "redimensionaCallback" como sendo a função de redimensionamento
34     #Isso significa que a função "redimensionaCallback" será chamada sempre que a janela for redimensionada,
35     #seja pelo sistema ou pelo usuário
36     glfw.set_window_size_callback(Window, redimensionaCallback)
37
38     #Define o contexto atual do GLFW como sendo a janela criada acima. O contexto define
39     #em qual janela o OpenGL irá funcionar, o que é essencial para que o programa funcione
40     glfw.make_context_current(Window)
41
42     #Buscamos informações a respeito do hardware (placa de vídeo) e a versão do OpenGL que a mesma dá suporte
43     print("Placa de vídeo: ", OpenGL.GL.glGetString(OpenGL.GL.GL_RENDERER))
44     print("Versão do OpenGL: ", OpenGL.GL.glGetString(OpenGL.GL.GL_VERSION))
45
```

Comentários no código

```
46 def inicializaObjetos():
47     global Vao
48     # Devido ao fato de que cada objeto que modelarmos possuir, geralmente, uma coleção
49     # de buffers de informações referentes aos seus vértices (tais como coordenadas dos vértices,
50     # coordenadas de texturas, normais, cores, etc), utilizamos um objeto do tipo Vertex Attribute Object (VAO)
51     # que "une" e "representa" todos os buffers do objeto em um único identificador.
52
53     # Nós devemos especificar um VAO para cada objeto que estamos modelando.
54     # No caso deste exemplo, vamos renderizar somente 1 triângulo, logo, precisamos somente de
55     # 1 VAO para representá-lo.
56
57     # Geramos o VAO, definindo um identificador para ele através glGenVertexArrays
58     Vao = glGenVertexArrays(1) #gera 1 único VAO
59     # Damos um bind no VAO, setando ele como VAO atual e colocando o mesmo no topo da máquina de estados do OpenGL
60     glBindVertexArray(Vao)
61
62     # Definição de um VBO para os vértices do triângulo
63     # - Primeiramente, definimos em um vetor de float os vértices do triângulo;
64     # - Em seguida, criamos uma cópia desses dados na placa gráfica através de uma unidade denominada Vertex Buffer Object (VBO).
65     # Para isso, nós geramos primeiramente um buffer vazio, através da função glGenBuffers, e então setamos esse buffer como buffer
66     # atual na máquina de estados do OpenGL através de glBindBuffer, e por fim copiamos os pontos para esse buffer através do glBufferData.
67     points = [
68         #X   Y   Z
69         0.0, 0.5, 0.0, #cima
70         0.5, -0.5, 0.0, #direita
71         -0.5, -0.5, 0.0 #esquerda
72     ]
73
74     # Convertamos o array do Python para um array da biblioteca NumPy, pois é o tipo de array que o OpenGL trabalha
75     points = np.array(points, dtype=np.float32)
76
77     pvbo = glGenBuffers(1) #cria 1 VBO
78     #Vertex Buffer Object (VBO):
79     #Propósito: O VBO é usado para armazenar os dados dos vértices.
80     #Funcionalidade: Ele armazena os dados do vértice, como posições, cores, normais, etc., em um buffer de memória na GPU.
81     #Uso Típico: Carrega os dados do vértice para a GPU usando glBufferData ou glBufferSubData.
82     #O VBO é, então, associado ao VAO usando glBindBuffer para que o VAO saiba de onde obter os dados.
83
84     glBindBuffer(GL_ARRAY_BUFFER, pvbo) #coloca o pvbo no topo da pilha/máquina de estados
85     glBufferData(GL_ARRAY_BUFFER, points, GL_STATIC_DRAW) #copia os dados do python para dentro do VBO
86     # Ativamos o primeiro atributo do VAO (índice 0), que é o atributo referente ao buffer das posições dos vértices.
87     glEnableVertexAttribArray(0)
88     # E então definimos o layout do buffer de vértices:
89     # - o primeiro parâmetro (0) significa que estamos definindo o layout do atributo 0 (buffer de vértices)
90     # - o segundo parâmetro (3) significa que esse buffer é formado por 3 variáveis (x,y, e z),
91     # - o terceiro parâmetro, indica que as variáveis são do tipo float
92     # - o quarto parâmetro indica que nós desejamos normalizar os valores
93     # - o quinto parâmetro é o byte offset entre os atributos, caso tenha sido especificado um único VBO para mais de um tipo de informação
94     # - o sexto parâmetro é o offset do primeiro elemento, que no nosso caso, é 0, pois queremos todos os elementos do array
95     # -- Devido a um bug da biblioteca, precisamos passar None ao invés de 0
96     glVertexAttribPointer(0, 3, GL_FLOAT, GL_FALSE, 0, None)
97
```

Comentários no código

```
46 def inicializaObjetos():
47     .....
48
49     # Definição de um VBO para as cores do triângulo. Observe que passamos como parâmetro o valor 1
50     # na chamada ao "glEnableVertexAttribArray", pois estamos ativando o segundo atributo deste VAO,
51     # que são as cores dos vértices. Além disso, também passamos o parâmetro 1 na chamada ao "glVertexAttribPointer",
52     # pois estamos definindo o layout do segundo atributo.
53     cores = [
54         #R    G    B
55         1.0, 0.0, 0.0, #vermelho
56         0.0, 1.0, 0.0, #verde
57         0.0, 1.0, 1.0 #azul
58     ]
59     cores = np.array(cores, dtype=np.float32) #converte o array para numpy
60     cvbo = glGenBuffers(1) #gera o vbo para as cores
61     glBindBuffer(GL_ARRAY_BUFFER, cvbo) #da um bind no vbo das cores
62     glBufferData(GL_ARRAY_BUFFER, cores, GL_STATIC_DRAW) #copia os dados para a memória de vídeo
63     glEnableVertexAttribArray(1) #ativa o índice 1 para o vbo das cores
64     glVertexAttribPointer(1, 3, GL_FLOAT, GL_FALSE, 0, None) #configura o vbo das cores
65
66 def inicializaShaders():
67     global Shader_programm
68     # Especificação do Vertex Shader:
69     # - O Vertex Shader é responsável por processar cada vértice individualmente na GPU.
70     # - A primeira linha especifica a versão da linguagem GLSL que estamos utilizando (4.0.0).
71     # - `layout(location = 0) in vec3 vertex_posicao`:
72     #   Essa variável de entrada (in) representa a posição de cada vértice,
73     #   que é enviada pela CPU via VBO (vertex buffer object).
74     # - `layout(location = 1) in vec3 vertex_cores`:
75     #   Essa variável de entrada representa a cor associada a cada vértice.
76     # - `out vec3 cores`:
77     #   Esta é uma variável de saída do vertex shader.
78     #   Ela serve para **passar a cor do vértice para o fragment shader**.
79     #   O OpenGL irá automaticamente interpolar esse valor entre os vértices ao longo da superfície.
80     # - Dentro da função `main()`, atribuímos a posição final do vértice à variável especial `gl_Position`,
81     #   que é obrigatória e define onde o vértice aparecerá na tela.
82     #   `gl_Position` deve ser um `vec4`, então adicionamos 1.0 como o componente `w` (homogêneo).
83     vertex_shader = """
84     #version 400
85     layout(location = 0) in vec3 vertex_posicao; //Vem do Python (IN), do VBO 0 (POSIÇÕES)
86     layout(location = 1) in vec3 vertex_cores; //Vem do Python (IN), do VBO 1 (CORES)
87     out vec3 cores;
88     void main () {
89         cores = vertex_cores;
90         gl_Position = vec4 (vertex_posicao.x, vertex_posicao.y, vertex_posicao.z, 1.0);
91     }
92     """
93
94     # Como os shaders são um programa "a parte", precisamos compilá-lo e verificar se não houve nenhum erro de compilação
95     vs = OpenGL.GL.shaders.compileShader(vertex_shader, GL_VERTEX_SHADER)
96     if not glGetShaderiv(vs, GL_COMPILE_STATUS):
97         infoLog = glGetShaderInfoLog(vs, 512, None)
98         print("Erro no vertex shader:\n", infoLog)
99
100
```

Comentários no código

```
# Especificação do Fragment Shader:
# - O Fragment Shader é executado para cada fragmento (pixel potencial) gerado durante a rasterização do objeto.
# - A primeira linha especifica a versão da linguagem GLSL utilizada (4.0.0).
# - `in vec3 cores`:
#   Essa variável de entrada recebe a **cor interpolada** dos vértices, vinda do vertex shader através da variável `out vec3 cores`.
#   O OpenGL automaticamente interpola os valores das cores ao longo dos fragmentos da superfície do triângulo.
# - `out vec4 frag_colour`:
#   Essa é a variável de saída do fragment shader. Ela define a **cor final** do pixel que será desenhado na tela.
#   Deve ser do tipo `vec4`, representando (R, G, B, A) – sendo `A` o canal de opacidade (alpha).
# - Dentro da função `main()`, atribuímos à `frag_colour` o valor da cor recebida, adicionando o valor de alpha como 1.0 (totalmente opaco).

fragment_shader = """
    #version 400
    in vec3 cores;
    out vec4 frag_colour;
    void main () {
        frag_colour = vec4 (cores.r, cores.g, cores.b, 1.0);
    }
"""

# Do mesmo modo que o vertex shader, precisamos compilar o fragment shader e verificar se não houve nenhum erro de compilação
fs = OpenGL.GL.shaders.compileShader(fragment_shader, GL_FRAGMENT_SHADER)
if not glGetShaderiv(fs, GL_COMPILE_STATUS):
    infoLog = glGetShaderInfoLog(fs, 512, None)
    print("Erro no fragment shader:\n", infoLog)

# Especificação do Shader Programm:
# Após compilarmos os shaders, precisamos combiná-los em um único programa, denominado GPU Shader Program.
# Para isso, chamamos a função compileProgram passando os dois shaders que irão formar o nosso shader program
# e testamos se não houve nenhum erro de linkagem
Shader_programm = OpenGL.GL.shaders.compileProgram(vs, fs)
if not glGetProgramiv(Shader_programm, GL_LINK_STATUS):
    infoLog = glGetProgramInfoLog(Shader_programm, 512, None)
    print("Erro na linkagem do shader:\n", infoLog)

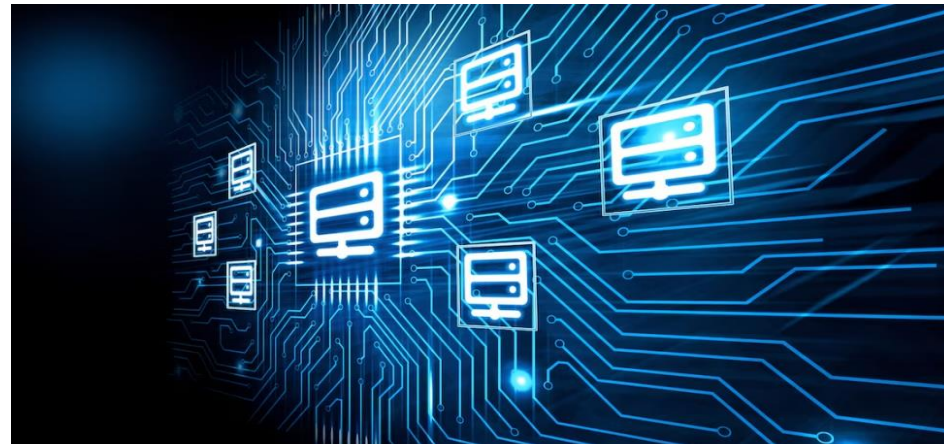
glDeleteShader(vs) #depois de copiados para GPU com shader podemos liberar a memoria com 'vs e 'fs'
glDeleteShader(fs)
```

Comentários no código

```
186 def inicializaRenderizacao():
187     global Window, Shader_programm, Vao, WIDTH, HEIGHT
188
189     # O triangulo é redesenhado o tempo todo, dentro de um laço de repetição
190     # que é executado enquanto a janela não for fechada
191     while not glfw.window_should_close(Window):
192         # Limpamos o buffer de cores da tela
193         glClear(GL_COLOR_BUFFER_BIT)
194         glClearColor(0.2, 0.3, 0.3, 1.0) #cor de fundo
195
196         # Redefinimos o tamanho da viewport para o tamanho atual da janela, a cada frame, de modo que
197         # o desenho se ajuste de acordo com o tamanho da tela
198         glViewport(0, 0, WIDTH, HEIGHT)
199
200         # Especificamos qual Shader Programm vamos utilizar
201         glUseProgram(Shader_programm)
202
203         # Setamos o objeto Vao como sendo o VAO atual na máquina de estados do OpenGL
204         glBindVertexArray(Vao)
205
206         # Desenhamos o triângulo especificado no vao
207         glDrawArrays(GL_TRIANGLES, 0, 3) #a partir do primeiro vértice, desenha 3 vértices
208
209         # Atualizamos outros eventos, tais como entradas pelo teclado, mouse, etc, caso ocorram
210         glfw.poll_events()
211
212         # Renderizamos na tela tudo aquilo que foi desenhado logo acima
213         glfw.swap_buffers(Window)
214
215         # Verificamos se a tecla ESC foi pressionada. Caso positivo, definimos que a tela deve ser
216         # fechada na próxima volta do laço.
217         # Para testar se outras teclas foram pressionadas, verifique o seguinte link:
218         # http://www.glfw.org/docs/latest/group\_input.html
219         if (glfw.PRESS == glfw.get_key(Window, glfw.KEY_ESCAPE)):
220             glfw.set_window_should_close(Window, True)
221
222     glfw.terminate()
223
224 # Função principal
225 def main():
226     inicializaOpenGL() #configuração do ambiente (janelas, bibliotecas, etc.)
227     inicializaObjetos() #modelagem dos objetos e envio dos mesmos para a placa de vídeo
228     inicializaShaders() #programação dos shaders, especificando como esses objetos devem ser renderizados
229     inicializaRenderizacao() #onde é feita a renderização na tela dos objetos modelados
230
231 if __name__ == "__main__":
232     main()
```

VAO x VBO

- VAO (Vertex Array Object) e VBO (Vertex Buffer Object) não são a mesma coisa em OpenGL, mas estão relacionados e são frequentemente usados juntos para gerenciar dados de vértices.



VAO (Vertex Array Object)

VAO (Vertex Array Object):

- Um VAO é um objeto que encapsula várias VBOs e configurações de atributos de vértices.
- Ele mantém o estado do vertex array, incluindo quais VBOs estão vinculados a quais atributos de vértices.
- O VAO permite definir o layout dos dados do vértice uma vez e, em seguida, alternar entre diferentes conjuntos de dados sem reconfigurar todos os atributos a cada renderização.
- Em resumo, enquanto o VBO armazena os dados dos vértices, o VAO gerencia o estado associado a esses dados, incluindo a configuração de atributos e a associação de VBOs específicos a esses atributos. O uso conjunto de VBOs e VAOs ajuda a organizar e otimizar o processo de renderização, especialmente quando você tem vários objetos com diferentes conjuntos de dados de vértices.

VBO - Vertex Buffer Object

VBO (Vertex Buffer Object):

- Um VBO é um buffer de memória que armazena dados dos vértices, como posições, cores, normais, etc.
- Ele é usado para armazenar as informações dos vértices de um objeto.
- O VBO é associado a um tipo específico de dado, como coordenadas de posição, cores, texturas, etc.
- Os dados são enviados para o VBO e podem ser usados em vários desenhos sem a necessidade de reenviar os dados a cada quadro.

VBO - Vertex Buffer Object

- VBO, ou Vertex Buffer Object, é uma parte essencial do processo de renderização em OpenGL. Ele é utilizado para armazenar dados de vértices, como posições, cores, normais e outras informações, diretamente na GPU (Unidade de Processamento Gráfico). Os VBOs são particularmente eficientes para transferir grandes quantidades de dados geométricos para a GPU, otimizando o desempenho da renderização.
- Ao usar VBOs em conjunto com shaders, você pode separar os dados de vértices do lado da CPU (Central Processing Unit) para o lado da GPU, permitindo que a GPU processe os vértices de forma paralela e acelerando o pipeline gráfico.

VBO - Vertex Buffer Object

Aqui estão algumas informações-chave sobre VBOs e sua relação com shaders:

Armazenamento de Dados:

- VBOs armazenam dados de vértices, como coordenadas espaciais (posições), cores, texturas, etc. Esses dados são usados pelos shaders durante o processo de renderização.

Separação de Vértices:

- Tradicionalmente, dados de vértices são fornecidos à GPU através de arrays de CPU. Com VBOs, você separa esses dados, colocando-os em buffers que residem na memória da GPU.

VBO - Vertex Buffer Object

Desempenho:

- O uso de VBOs melhora o desempenho porque os dados estão armazenados na GPU, eliminando a necessidade de transferências frequentes entre a CPU e a GPU durante o processo de renderização.

Shader Atributos:

- Os dados armazenados em VBOs são associados a atributos nos shaders. Por exemplo, você pode ter um buffer de posição, um buffer de cor, etc., que são vinculados a variáveis de atributo no vertex shader.

VBO - Vertex Buffer Object

GLSL Atributos em Shaders:

- Os atributos nos shaders, como `layout (location = 0) in vec3 vertex_posicao;`, indicam onde os dados dos VBOs serão lidos no shader. Nesse exemplo, os dados de posição do vértice são lidos no atributo de localização 0.

- **Desenho Eficiente:**

Ao usar VBOs, você pode usar funções como `glDrawArrays` ou `glDrawElements` para renderizar diretamente a partir dos dados do VBO, sem a necessidade de transferir os dados de volta para a CPU.

VBO - Vertex Buffer Object

Atualização Dinâmica:

- VBOs suportam atualizações dinâmicas, permitindo que você altere partes específicas dos dados de vértices sem recriar todo o buffer.
- Em resumo, VBOs são uma ferramenta poderosa para otimizar a transferência e uso eficiente de dados de vértices na GPU, melhorando significativamente o desempenho de aplicações gráficas em OpenGL.

Exercícios:

Exercício 1 – Trocar a cor do triângulo

Objetivo: Alterar as cores dos vértices do triângulo para outras combinações.

Exercício 2 – Adicionar um segundo triângulo

Objetivo: Adicionar um novo triângulo com novas cores, no mesmo shader e mesma janela, utilizando o mesmo VAO e VBO (mesmo array para pontos e cores). **OBS:** Lembre-se, se for utilizar o mesmo VAO, agora são 6 vértices para desenhar: `glDrawArrays(GL_TRIANGLES, 0, ?)`

Exercício 3 – Inverter um triângulo na vertical

Objetivo: Alterar um dos triângulos para que ele fique de cabeça para baixo (qual eixo inverter o sinal dos valores?).

Envie um print e o código (formato zip) na atividade de hoje, separadas por pastas, e um arquivo com comentários do que foi necessário alterar no código para realizar as alterações. Pode ser a parte do código adicionado mais comentários.

Exercícios:

Exercício 4 – Adicionar um segundo triângulo e usar 2 VAO e 4 VBOS (1 para pontos e 1 para cores do triângulo 01, 1 para pontos e 1 para cores para o triângulo 02)

Objetivo: Adicionar um novo triângulo, porém cada um com seu VAO e VBOS para pontos e cores)

Exercício 5 – Gerar um quadrado através de dois triângulos, usando `glDrawArrays(GL_TRIANGLES, 0, 3)`

Objetivo: Adicionar um quadrado com dois triângulos, pode ser usando o mesmo vao.

Envie um print e o código (formato zip) na atividade de hoje, separadas por pastas, e um arquivo com comentários do que foi necessário alterar no código para realizar as alterações. Pode ser a parte do código adicionado mais comentários.

Exercícios:

Exercício 6 – Adicionar um quadrado e um triângulo lado a lado, utilizando duas funções separadas para criar cada forma e usando 2 shaders e seus respectivos VAOs (um para cada objeto).

Objetivo: Adicionar um quadrado e um triângulo lado a lado, utilizando 2 Shaders.

DICAS:

#desenha o quadrado

```
glUseProgram(Shader_programm_quadrado) #ativa o shader do quadrado
```

```
glBindVertexArray(Vao_quadrado) #ativa o VAO do quadrado
```

```
glDrawArrays(GL_TRIANGLES, 0, 6) #renderiza o quadrado
```

#desenha o triângulo

```
glUseProgram(Shader_programm_triangulo) #troca de shader, ativando o shader do triangulo
```

```
glBindVertexArray(Vao_triangulo) #troca de VAO, ativando o VAO do triângulo
```

```
glDrawArrays(GL_TRIANGLES, 0, 3) #renderiza o triângulo
```

Função principal

```
def main():
```

```
    inicializaOpenGL()
```

```
    inicializaQuadrado()
```

```
    inicializaTriangulo()
```

```
    inicializaShaderQuadrado()
```

```
    inicializaShaderTriangulo()
```

```
    inicializaRenderizacao()
```

```
if __name__ == "__main__":
```

```
    main()
```

Exercícios:

Exercício 7 – Analise os códigos abaixo disponibilizados na aula de hoje, e faça um resumo para entregar na próxima aula, junto com os códigos dos exercícios anteriores (códigos Ex1 a Ex6).

Códigos disponíveis para analisar e fazer o resumo:

a) Ex5_Transformações_Geométricas.py

b) Ex6_Câmera_e_objetos.py

Explique as principais partes do código, transformações e como esses dados são usados pela GPU. No exercício 7 não é necessário entregar código, somente o resumo.

Referências e material de apoio

Material do Professor Guilherme Chagas Kurtz, 2023.

GOMES, Jonas; VELHO, Luiz. Computação gráfica. Rio de Janeiro: Impa, 1998.

HEARN, Donald; Baker, M. Pauline. Computer graphics: C version. London: Prentice Hall, 1997.

HETEM JUNIOR, Annibal. Computação gráfica. Rio de Janeiro, RJ: LTC, 2006. 161 p. (Coleção Fundamentos de Informática).

HILL Jr, Francis S. Computer graphics using open GL. New Jersey: Prentice Hall, 2001.

WATT, Alan. 3D computer graphics. Harlow: Addison-Wesley, 2000

Thank you for your attention!!



Email: andre.flores@ufn.edu.br